
Compte rendu n°5
Tp Tests de Logiciels
Les Tests d'Intégration en Java
Behaviour-Driven Development avec Cucumber-JVM

Nom:Rayen Bouyahia
Classe:L2CS01

1. Introduction

Ce TP a pour objectif de mettre en place un environnement de développement orienté comportement (BDD – Behaviour-Driven Development) en Java, en utilisant le framework Cucumber-JVM couplé à JUnit. L'approche BDD permet de lier directement les spécifications fonctionnelles (rédigées en langage naturel Gherkin) à du code de test exécutable.

L'environnement de travail utilisé est IntelliJ IDEA Community Edition avec JDK 1.8 et Maven comme gestionnaire de dépendances.

2. Création du projet Maven

La première étape consiste à créer un nouveau projet Maven dans IntelliJ IDEA en utilisant l'archétype standard :

- Ouvrir IntelliJ et sélectionner « Nouveau projet »
- Choisir l'archétype maven-archetype-quickstart
- Renseigner le GroupId : com.example et l'ArtifactId : exemple
- Donner le nom « Exemple » au projet et laisser IntelliJ gérer l'emplacement

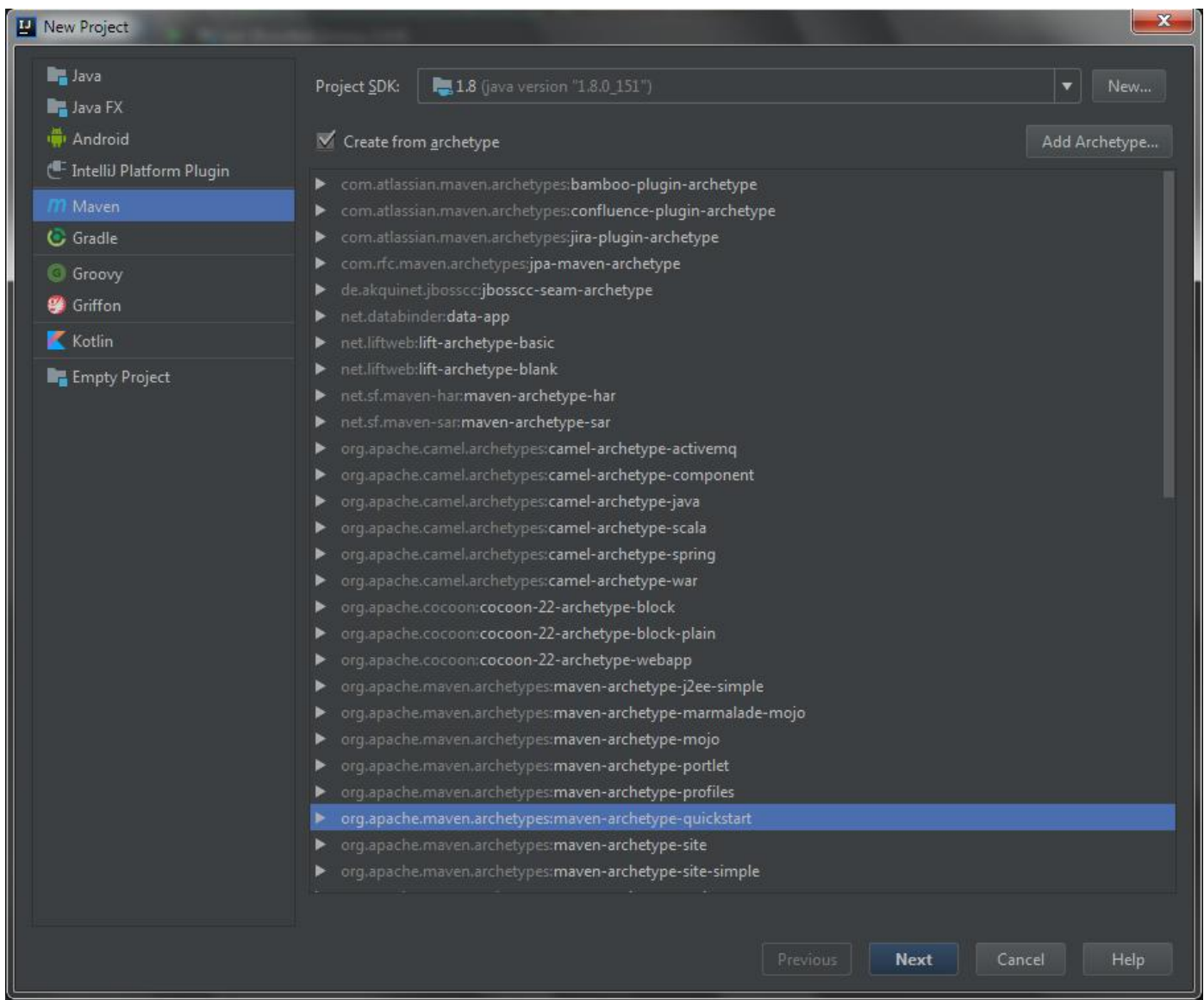


Figure 1 — Sélection de l'archétype Maven dans IntelliJ

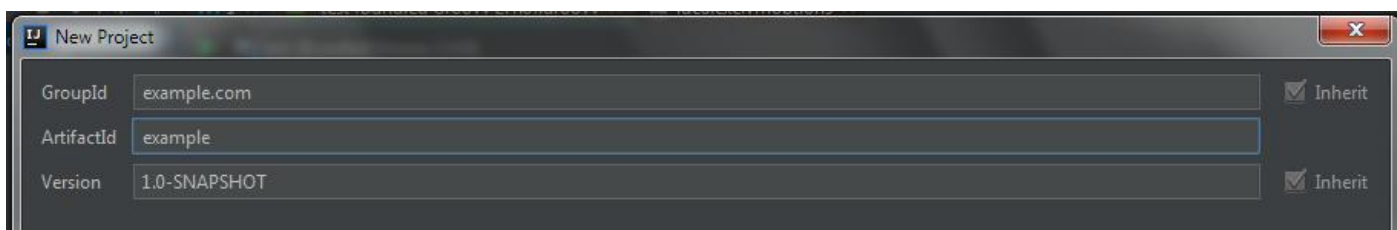


Figure 2 — Saisie du GroupId et de l'ArtifactId

3. Configuration des dépendances (pom.xml)

Les bibliothèques suivantes ont été ajoutées au fichier pom.xml :

- junit:junit:4.11 — framework de tests unitaires
- info.cukes:cucumber-java:1.1.3 — intégration Java de Cucumber
- info.cukes:cucumber-junit:1.1.3 — runner Cucumber pour JUnit

Le dépôt Sonatype Snapshots a également été ajouté pour permettre le téléchargement des versions snapshot de Cucumber.

Extrait du fichier pom.xml :

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.11</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>info.cukes</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>1.1.3</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>info.cukes</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>1.1.3</version>
  <scope>test</scope>
</dependency>
```

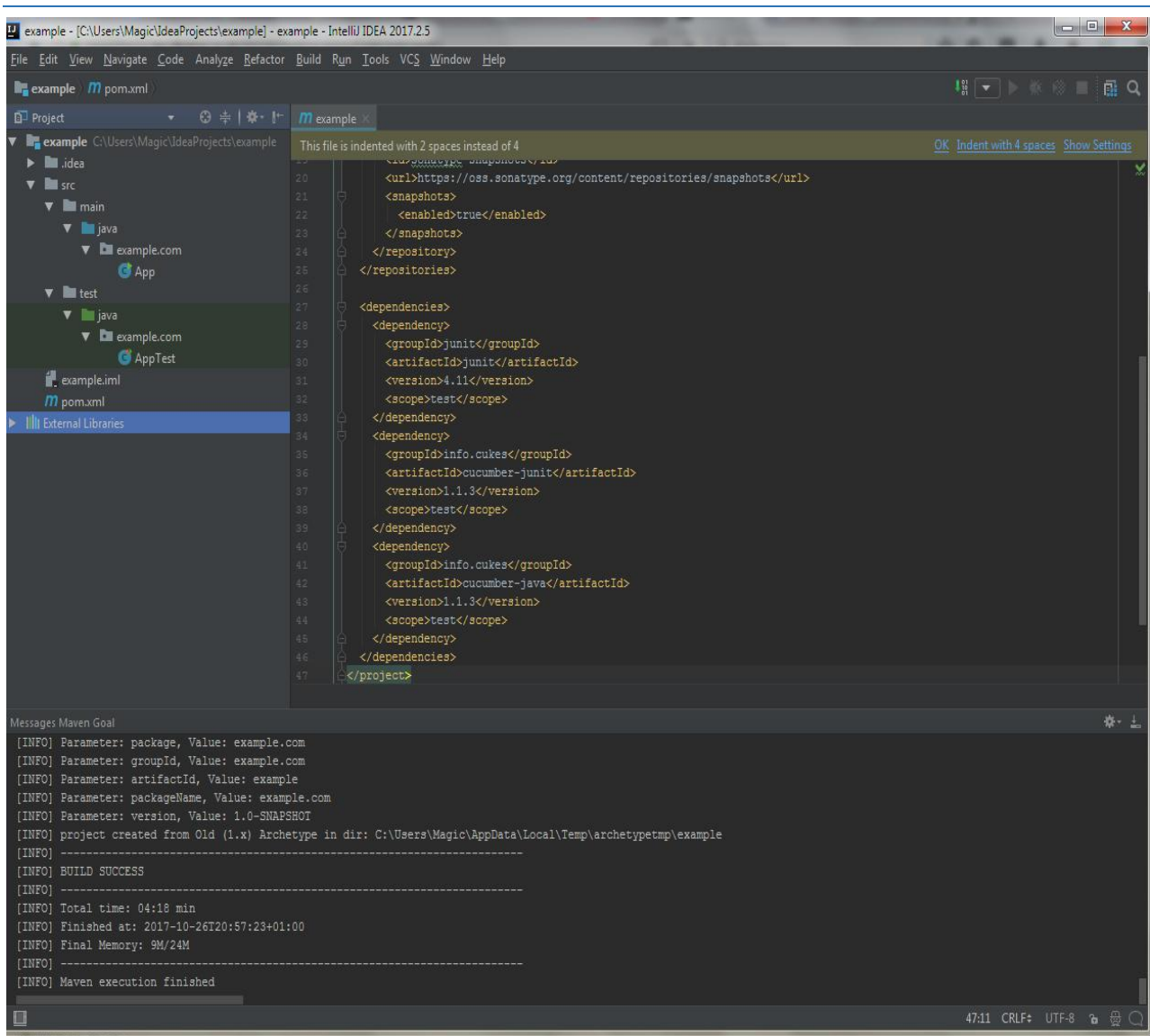


Figure 3 — Dépendances Maven téléchargées avec succès (External Libraries)

4. Structure du projet et fichier feature

Après l'import des dépendances, il faut créer le dossier `src/test/resources` et le marquer comme « Test Resources Root » dans IntelliJ. Ce répertoire accueille les fichiers `.feature` contenant les scénarios Gherkin.

Le fichier `Example.feature` définit le scénario suivant :

```
Feature: Fizz Buzz Game
  So that plays can be validated
  As a Fizz Buzz umpire
  I want to enter a play and see the correct answer

Scenario: Get answers based on played numbers
  Given I am officiating a FizzBuzz game
  When the number 1 is played
  Then I should be told the correct answer is "1"
```

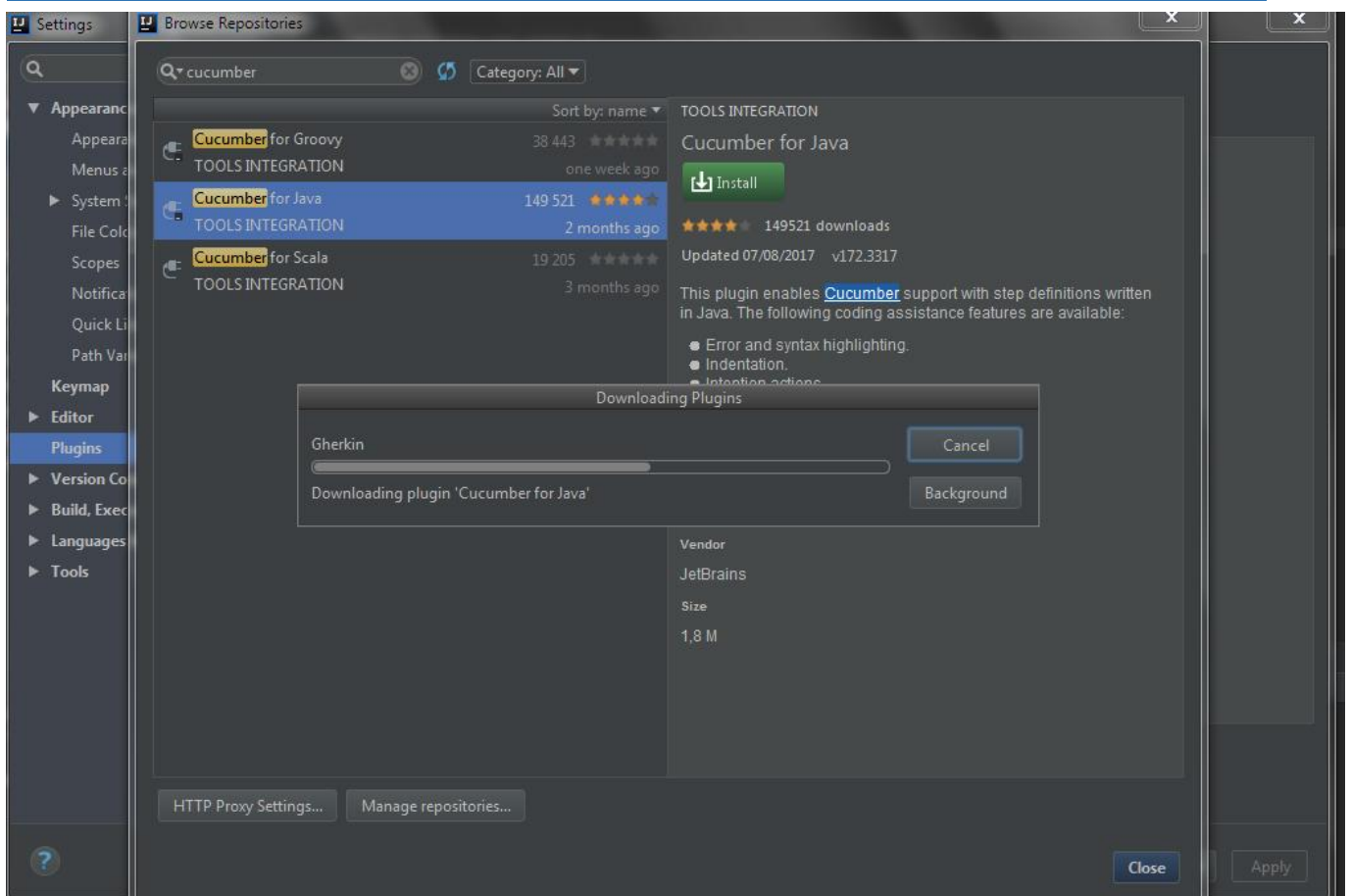


Figure 4 — Marquage du répertoire resources comme Test Resources Root

5. Implémentation des Step Definitions

Une fois le fichier feature exécuté via le CukesRunner, Cucumber génère automatiquement les squelettes de méthodes manquantes. Ces snippets sont ensuite implémentés dans la classe ExampleSteps.java.

Classe ExampleSteps.java :

```
@Given("^I am officiating a FizzBuzz game$")
public void I_am_officiating_a_FizzBuzz_game() {
    _target = new Example();
}

@When("^the number (\\d+) is played$")
public void the_number_is_played(int playedNumber) {
    _actualResult = _target.checkPlay(playedNumber);
}

@Then("^I should be told the correct answer is \"([^\"]*)\"$")
public void I_should_be_told_the_correct_answer_is(String expectedResult) {
    assertEquals(expectedResult, _actualResult);
}
```

Classe métier Example.java :

```
public class Example {
```

```

public String checkPlay(int playedNumber) {
    return Integer.toString(playedNumber);
}
}

```

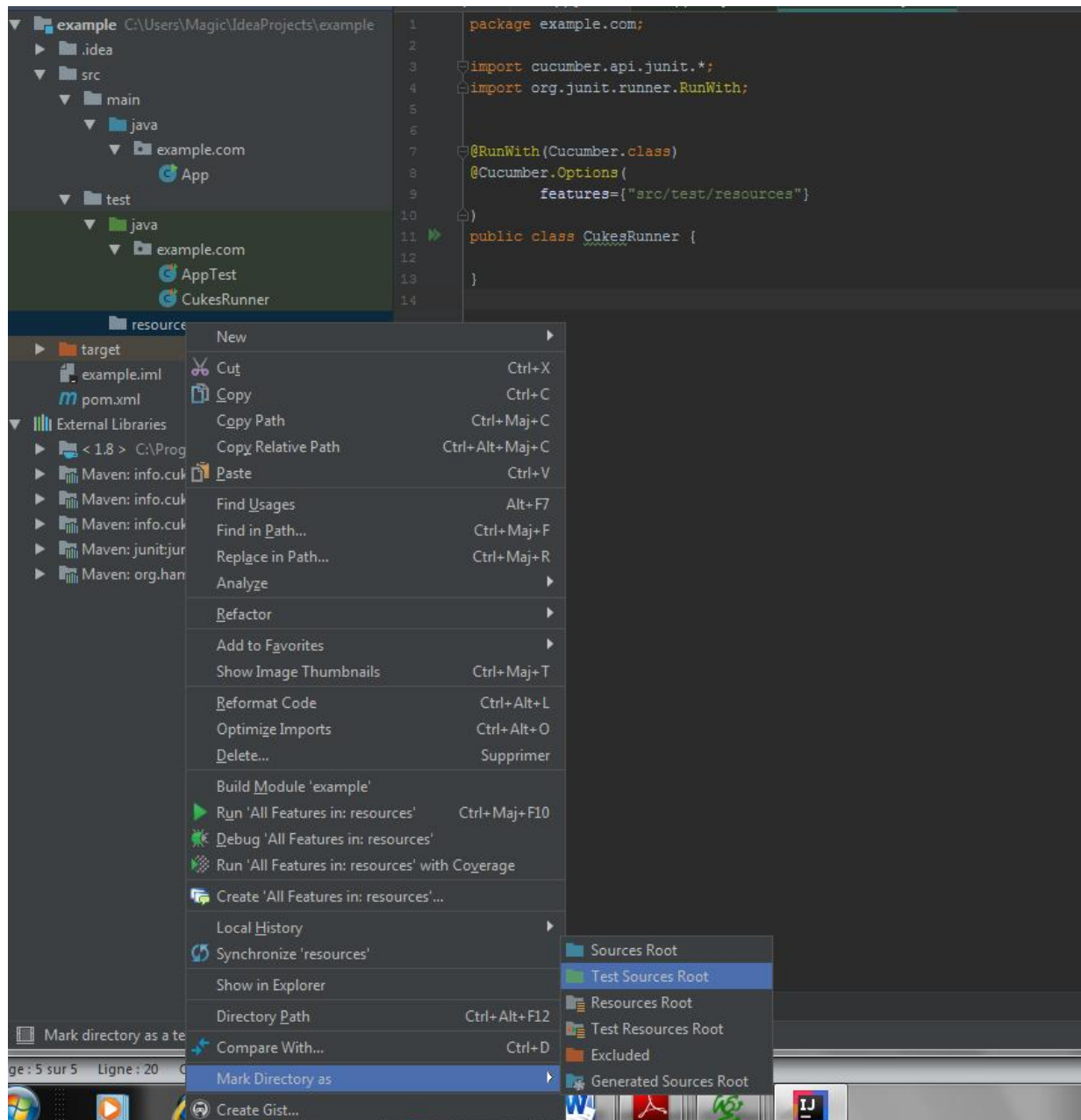


Figure 5 — Structure finale du projet et résultat d'exécution des tests (All Tests Passed)

6. Résultats et conclusion

Après l'implémentation des step definitions et de la classe métier, l'exécution du CukesRunner produit un résultat positif : tous les tests passent (couleur verte, « All Tests Passed »).

Ce TP m'a permis de comprendre la mise en place d'une démarche BDD dans un projet Java avec Maven. Le principal apport est la traçabilité directe entre les spécifications rédigées en Gherkin et le code de test, ce qui facilite la communication entre développeurs et parties prenantes non techniques.

Points clés retenus :

- Le fichier .feature joue le rôle de documentation vivante et exécutable
- Cucumber génère les squelettes de méthodes manquantes, ce qui accélère l'implémentation
- La configuration Maven centralise toutes les dépendances nécessaires
- Le CukesRunner fait le lien entre les fichiers .feature et les classes Java

Rayen Bouyahia — L2CS01